(in the form of individual computers added to the network by users) as they become available.

[3] http://setiathome.ssl.berkeley.edu/

For any given multithreaded program, the number of threads that are performing useful work will vary as the program runs. Even if every thread is doing useful work for the entirety of its existence, the application may initially have only one thread, which will then have the task of spawning all the others. But even that's a highly unlikely scenario. Threads often spend time waiting for each other or waiting for I/O operations to complete.

Every time one thread has to wait for something (whatever that something is), unless there's another thread ready to take its place on the processor, you have a processor sitting idle that could be doing useful work.

A simplified way of looking at this is to divide the program into "serial" sections where only one thread is doing any useful work and "parallel" sections where all the available processors are doing useful work. If you run your application on a system with more processors, the "parallel" sections will theoretically be able to complete more quickly, because the work can be divided between more processors, whereas the "serial" sections will remain serial. Under such a simplified set of assumptions, you can therefore estimate the potential performance gain to be achieved by increasing the number of processors: if the "serial" sections constitute a fraction f_S of the program, then the performance gain P from using N processors can be estimated as

$$P = \frac{1}{f_S + \frac{1 - f_S}{N}}$$

This is *Amdahl's law*, which is often cited when talking about the performance of concurrent code. If everything can be parallelized, so the serial fraction is 0, the speedup is simply N. Alternatively, if the serial fraction is one third, even with an infinite number of processors you're not going to get a speedup of more than 3.